Intermediate Python Programming — Practice Questions

Core skills that carry straight into Machine Learning | No OOP

This set contains 25 intermediate-level Python questions with full answers and example code. Each question sticks to core Python (no classes or OOP) and is followed by a short note on how the same idea shows up later in Machine Learning, so the transition feels natural.

Q1. What is a list comprehension, and how would you use one to create a list of squares from 1 to 10?

Answer:

A list comprehension is a compact way to build a list in a single line using the pattern [expression for item in iterable]. It replaces a longer for-loop with append() calls.

Code:

```
squares = [x**2 for x in range(1, 11)]
print(squares)
```

ML Connection: List comprehensions are used constantly to transform raw data — for example, extracting or reshaping a single feature column before it is fed into a model.

Q2. How would you filter a list using a list comprehension to keep only the even numbers from 1 to 20?

Answer:

You add an 'if' condition after the loop inside the comprehension. Only items that satisfy the condition are included in the resulting list.

Code:

```
evens = [x for x in range(1, 21) if x % 2 == 0]
print(evens)
```

ML Connection: This filtering pattern mirrors how you would drop invalid samples or keep only rows that meet a labeling condition before training a model.

Q3. What does a nested list comprehension do? Write one that flattens a 2D matrix into a 1D list.

Answer:

A nested list comprehension places one comprehension's for-loop inside another, letting you iterate over an outer list and then an inner list in a single expression.

Code:

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [num for row in matrix for num in row]
print(flat)
```

ML Connection: Flattening nested structures like this is common when converting a 2D array (such as an image patch or a batch of samples) into a single 1D feature vector.

Q4. What is a lambda function? Use one to sort a list of (name, score) tuples by score in descending order.

Answer:

A lambda is a small, anonymous, one-line function, useful when you need a quick function as an argument (for example, as a sort key) without formally defining one with 'def'.

Code:

```
students = [("Ram", 72), ("Sita", 88), ("Hari", 65)]
sorted_students = sorted(students, key=lambda x: x[1], reverse=True)
print(sorted_students)
```

ML Connection: Lambda-based sorting is exactly how you rank model predictions by confidence score or rank features by importance.

Q5. Explain the map() function with an example that converts a list of Celsius temperatures to Fahrenheit.

Answer:

map(function, iterable) applies a function to every item of an iterable and returns an iterator of the results, which you can convert to a list.

Code:

```
celsius = [0, 20, 37, 100]
fahrenheit = list(map(lambda c: (c * 9 / 5) + 32, celsius))
print(fahrenheit)
```

ML Connection: map() is the manual version of applying a transformation, like scaling or unit conversion, across an entire feature column.

Q6. Explain the filter() function and use it to keep only the positive numbers from a list.

Answer:

filter(function, iterable) keeps only the items for which the function returns True, discarding everything else.

Code:

```
nums = [-5, 3, -1, 8, 0, -2, 9]
positives = list(filter(lambda x: x > 0, nums))
print(positives)
```

ML Connection: filter() mirrors how you would remove invalid, missing, or out-of-range data points from a dataset during cleaning.

Q7. What does functools.reduce() do? Use it to compute the product of all numbers in a list.

Answer:

reduce(function, iterable) repeatedly applies a function to pairs of items, collapsing the whole iterable down into a single cumulative value.

Code:

```
from functools import reduce
nums = [1, 2, 3, 4, 5]
product = reduce(lambda a, b: a * b, nums)
print(product)
```

ML Connection: reduce() generalizes aggregation, similar to combining the loss from many batches into one total loss value during training.

Q8. What is the difference between *args and **kwargs in a function definition? Write a function that accepts both.

Answer:

*args collects any extra positional arguments into a tuple, while **kwargs collects any extra keyword arguments into a dictionary. Both let a function accept a flexible number of inputs.

Code:

```
def describe_model(*layers, **hyperparams):
    print("Layers:", layers)
    print("Hyperparameters:", hyperparams)

describe_model(64, 32, 16, learning_rate=0.01, epochs=50)
```

*ML Connection: Many ML library functions (model constructors, training loops) use *args/**kwargs to accept a variable number of layers or hyperparameters.*

Q9. What does the zip() function do? Use it to pair feature values with their corresponding labels.

Answer:

zip() combines two or more iterables element by element, producing tuples that pair up values from each iterable at the same position.

Code:

```
features = [5.1, 4.9, 4.7]
labels = ["setosa", "setosa", "versicolor"]
paired = list(zip(features, labels))
print(paired)
```

ML Connection: This is precisely how features (X) are paired with labels (y) row by row when a dataset is assembled for training.

Q10. What does enumerate() do, and why is it useful in a for loop? Print each fruit with its index.

Answer:

enumerate() adds a running counter to an iterable, so a for loop can access both the index and the value without maintaining a separate counter variable.

Code:

```
fruits = ["apple", "banana", "cherry"]
for idx, fruit in enumerate(fruits):
    print(idx, fruit)
```

ML Connection: enumerate() is useful for tracking the row or sample index while looping through a dataset during preprocessing or batching.

Q11. Write a dictionary comprehension that maps each word in a list to its length.

Answer:

A dictionary comprehension follows the pattern {key_expr: value_expr for item in iterable}, building a dictionary in a single line.

Code:

```
words = ["data", "model", "train", "predict"]
word_lengths = {w: len(w) for w in words}
print(word_lengths)
```

ML Connection: Comprehensions like this often build lookup tables, such as mapping class names to numeric labels for classification tasks.

Q12. What is a generator function, and how does it differ from a normal function? Write a generator that yields Fibonacci numbers.

Answer:

A generator uses 'yield' instead of 'return' to produce values one at a time, pausing its state between calls. This makes it memory-efficient because values are produced lazily instead of all at once.

Code:

```
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b

print(list(fibonacci(8)))
```

ML Connection: Generators matter when loading large datasets that do not fit in memory — ML data loaders stream batches lazily using this same idea.

Q13. How do you handle exceptions in Python? Write code that safely divides two numbers and handles a ZeroDivisionError.

Answer:

try/except blocks let a program catch specific errors and respond gracefully instead of crashing when something goes wrong.

Code:

```
def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        print("Cannot divide by zero")
        return None
```

ML Connection: Exception handling protects data pipelines — for example, skipping a corrupted row instead of crashing an entire training run.

Q14. What are f-strings, and how do you use one to format a floating point number to two decimal places?

Answer:

f-strings (formatted string literals) let you embed expressions directly inside a string using curly braces, with optional formatting instructions after a colon.

Code:

```
accuracy = 0.94567
print(f"Model accuracy: {accuracy:.2f}")
```

ML Connection: f-strings are the standard way to neatly print metrics such as accuracy, loss, or precision while evaluating a model.

Q15. How do you read data from a CSV file into a list of rows using Python's built-in csv module?

Answer:

The csv module provides a reader object that iterates over each row of a CSV file, returning each row as a list of string values.

Code:

```
import csv

with open("data.csv", "r") as f:
    reader = csv.reader(f)
    rows = [row for row in reader]
```

ML Connection: Almost every ML project starts by loading a CSV dataset this way — it is the foundation before moving on to pandas.read_csv().

Q16. How do you read and write JSON data in Python?

Answer:

The json module's dump() and dumps() functions convert Python objects to JSON, while load() and loads() parse JSON back into Python objects.

Code:

```
import json

data = {"model": "linear_regression", "accuracy": 0.87}
with open("result.json", "w") as f:
    json.dump(data, f)

with open("result.json") as f:
    loaded = json.load(f)
```

ML Connection: JSON is widely used to store model configurations, hyperparameters, and evaluation results between training runs.

Q17. Write a recursive function to compute the factorial of a number.

Answer:

A recursive function calls itself with a smaller input until it reaches a base case that stops the recursion.

Code:

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

ML Connection: Recursive thinking is useful for understanding tree-based models, such as decision trees, which are built by recursively splitting data.

Q18. What is a decorator in Python? Write a simple decorator that prints how long a function takes to run.

Answer:

A decorator is a function that wraps another function to add extra behavior (such as logging or timing) without modifying the original function's code.

Code:

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.time() - start:.4f}s")
        return result
    return wrapper

@timer
def train_model():
    time.sleep(1)
```

ML Connection: Decorators like this are commonly used to measure training time per epoch or to log each step of an experiment automatically.

Q19. How do you remove duplicate values from a list using a set? Show an example.

Answer:

Converting a list to a set automatically removes duplicates because sets only store unique elements; converting it back to a list gives a de-duplicated list.

Code:

```
values = [1, 2, 2, 3, 4, 4, 4, 5]
unique_values = list(set(values))
```

```
print(unique_values)
```

ML Connection: Removing duplicate records is a common data-cleaning step before training, since repeated rows can bias a model's learned patterns.

Q20. What is tuple unpacking? Show how to unpack (feature, label) tuples inside a for loop.

Answer:

Tuple unpacking assigns each element of a tuple to its own variable in one statement, which is especially convenient inside a for loop over paired data.

Code:

```
dataset = [(5.1, "yes"), (3.4, "no"), (7.2, "yes")]
for feature, label in dataset:
    print(feature, label)
```

ML Connection: This pattern appears constantly when looping through paired (X, y) training examples in a dataset.

Q21. How do you sort a list of dictionaries by a specific key? Sort student records by 'score'.

Answer:

sorted() accepts a key function; passing a lambda that returns the desired dictionary value lets you sort a list of dictionaries by any field.

Code:

```
students = [{"name": "Ram", "score": 72}, {"name": "Sita", "score": 88}]
sorted_students = sorted(students, key=lambda s: s["score"], reverse=True)
```

ML Connection: The same sorting logic is used to rank model outputs, such as sorting predictions by their probability score.

Q22. What is slicing in Python, and how would you use it to split a list into two halves?

Answer:

Slicing (list[start:end]) extracts a sub-portion of a sequence. Using the midpoint index as the boundary splits a list cleanly into two parts.

Code:

```
data = list(range(1, 11))
mid = len(data) // 2
```

```
first_half = data[:mid]
second_half = data[mid:]
```

ML Connection: Slicing is essentially the manual version of what train_test_split() does automatically in libraries like scikit-learn.

Q23. How do you compute the mean, median, and standard deviation of a list using Python's built-in statistics module?

Answer:

The statistics module provides ready-made functions — mean(), median(), and stdev() — for common summary statistics, without needing an external library.

Code:

```
import statistics as stats

scores = [72, 88, 65, 90, 55]
mean = stats.mean(scores)
median = stats.median(scores)
std_dev = stats.stdev(scores)
```

ML Connection: These are the same summary statistics computed during exploratory data analysis (EDA), before any model is trained.

Q24. How would you normalize a list of numbers to a 0-1 range manually using min-max scaling, without external libraries?

Answer:

Min-max scaling subtracts the minimum value and divides by the range (maximum minus minimum), so every value ends up between 0 and 1.

Code:

```
data = [10, 20, 30, 40, 50]
minimum, maximum = min(data), max(data)
normalized = [(x - minimum) / (maximum - minimum) for x in data]
```

ML Connection: Min-max scaling is a standard preprocessing step so that large-valued features do not unfairly dominate a model's learning process.

Q25. How do you manually split a dataset into training and testing sets (an 80/20 split) using basic Python, without scikit-learn?

Answer:

Shuffle the data to remove ordering bias, then slice it at the 80% mark so the first portion becomes the training set and the remainder becomes the test set.

Code:

```
import random

data = list(range(1, 101))
random.shuffle(data)
split_index = int(0.8 * len(data))
train_set = data[:split_index]
test_set = data[split_index:]
```

ML Connection: This is exactly the logic behind train_test_split() in scikit-learn — understanding it manually makes it much easier to spot data leakage later on.